

ooRexx/WASM and JavaScript

October 8, 2026 · Josep Maria Blasco

What this is

This document specifies a way for ooRexx programs to use JavaScript, both in the browser and under Node: `.JSObject`, which gives access to all of JavaScript as Rexx objects. All of it is implemented and can be tried in an online playground, rexw.epbcn.com/ooRexx-wasm. It is offered for discussion.

Any JavaScript object, function, array or Promise reaches Rexx as a `.JSObject`, by reference, and is used with ordinary messages. The way in is `.js`, which `::requires "js.cls"` defines: it holds everything JavaScript knows by name, such as `Math`, `Map` or `fetch` (in JavaScript terms, the global object).

```
say .js~Math~max(3, 7)           -- 7
m = .js~Map~new; m~set("a", 1); say m~size    -- 1
resp = .js~fetch("data.json")~await         -- a Promise, waited for
doc = .js~page~document          -- the browser page's DOM

p = doc~createElement("p")
p~textContent = "Hola"
doc~body~appendChild(p)

/* ... */
::requires "js.cls"
```

Realms

JavaScript always runs in a "realm", a world with its own global objects. In the browser, ooRexx does not run in the page itself but in a background thread (a Web Worker), so that a long Rexx program never freezes the page. That thread is one realm, `.js`: it has the network (`fetch`), the browser's local database and WebSockets, but cannot touch what is on the screen. What the user sees, the DOM, belongs to another realm, the page's realm, `.js~page`. A JavaScript object of one realm cannot be given to the other (as an argument, or assigned to a property): strings, numbers and booleans travel as values, objects only within their realm.

Under Node there is a single realm, `.js`, which also gives Node's modules (`.js~require("node:fs")`).

In the browser, the page that runs ooRexx must grant each realm (`js: {worker: true, page: true}`). A grant gives the program everything JavaScript can do in that realm.

Messages

`o~name` reads a property, or calls it when it is a function (a constructor, such as `Map`, is returned instead, so that `.js~Map~new` works); `o~name(a, b)` calls a method; `o~name = v` sets a property that exists. ooRexx uppercases message names, so they are resolved case-

insensitively along the prototype chain. `o["name"], o["name"] = v` (which also creates the property) and `o~invoke("name", ...)` are exact.

Method	Does
<code>~new(args...)</code>	<code>new o(args...)</code>
<code>~await</code>	waits for a Promise and gives its value; a rejection is an error
<code>~typeof</code> , <code>~className</code> , <code>~instanceOf(c)</code>	JavaScript's <code>typeof</code> , the class name, <code>instanceof</code>
<code>~hasProperty(name)</code> , <code>~propertyNames</code>	JavaScript's <code>in</code> , with the exact name; <code>Object.keys</code>
<code>~makeArray</code> , <code>~supplier</code>	for <code>DO x OVER o</code> and <code>DO WITH INDEX i ITEM v OVER o</code> : arrays, Maps, Sets and plain objects
<code>~byteString</code>	an ArrayBuffer's or typed array's bytes as a Rexx string

Values

JavaScript strings, numbers, booleans (1/0) and bigints come back as Rexx strings; null and undefined as `.nil`. Rexx strings always go as strings, so "007" stays "007": `.js~num(x)`, `.js~bool(x)`, `.js~bigint(x)` and `.js~bytes(x)` (a Uint8Array) force a type, and `.js~true`, `.js~false` and `.js~undefined` are those values. `.nil` goes as null. An Array goes as an array and a Directory or StringTable as a plain object, both as copies; `.js~newArray(items...)` and `.js~newObject(directory)` make them in JavaScript instead, held by reference.

Errors

A JavaScript exception is SYNTAX 98.900, "JavaScript error: TypeError: ...", with what was thrown in `condition("0")~additional[2]`.

Events

JavaScript cannot call into Rexx while Rexx is running, so a callback is a `.JSHandler`, made with `.js~handler(object, message [, options])`. JavaScript gets it as a function that only queues the call and returns at once; a Rexx thread takes the calls when it is ready:

```
btn~addEventListener("click", .js~handler(self, "CLICKED", "preventDefault"))
.js~eventLoop                -- sends CLICKED for each call
```

`.js~eventLoop` sends the message to the object, with the call's arguments, for each call until `.js~stopEventLoop`, from a handler or another thread. `.js~nextEvent([seconds])` takes one event instead (a `.JSEvent`, which `~dispatch` sends; `.nil` when the time is over), so a program can run its own loop. Several threads may wait; each event goes to one.

Because the function returns before Rexx sees the call, it cannot answer anything (no sort comparators), and by then the browser has finished dispatching the event (`event~currentTarget` is `.nil`). What must happen during the dispatch goes in the options: `preventDefault`, `stopPropagation`, `stopImmediatePropagation`. Option `latest` keeps only the newest event of the handler waiting, for mousemove or scroll.

A handler stays alive from its first trip to JavaScript until `handler~release` or the end of the run.

Pitfalls

- **Case-insensitive names can pick the wrong property.** `.js~page~Event` finds `window.event`, the current event, before `window.Event`, the constructor. Use the exact form: `w["Event"]~new("input")`.
- **Rexx methods win.** The methods in the table above, and those of Rexx's Object (`send`, `start`, `copy`, `run`, ...), are not passed on to JavaScript. `el~className` names the JavaScript class (`HTMLDivElement`), not the element's `class` attribute: use `el["className"]`, and `o~invoke("send", x)`.
- **Directory keys are upper case.** `d~method = "POST"` stores `METHOD`, which a JavaScript options object does not know: use `d["method"] = "POST"`.

Relation to BSF4ooRexx and OLEObject

The design follows Rony Flatscher's BSF4ooRexx, moved from Java to JavaScript: `.JSObject` is to JavaScript objects what `BSF.CLS` is to Java objects. Messages work as with OLEObject on Windows: names are case-insensitive, and `o~name` reads a property or calls a method.

Questions to decide

1. **Names.** Case-insensitive resolution makes `doc~getElementById` work as written, at the price of the rare wrong pick shown above. Would you rather have exact names only?
2. **Values.** Rexx strings always go as strings, and a number is forced with `.js~num`. Would you rather have numbers converted automatically, at the price of "007" becoming 7?
3. **Events.** JavaScript's callbacks become queued calls, taken by `.js~eventLoop` or `.js~nextEvent`. Does this model suit you, or would you expect another?

Replies on this list are very welcome.